



MANUAL TEST GUIDE · MODULE 02 OF 11

Rate Management Functional-Flow Test Walkthroughs

Scenario-driven, hands-on QA guide for the **logistics.pricing** Rate Management System (RMS) — the buy/sell rate lifecycle, the margin & predictive-margin engine, maker-checker approval, spot RFQs, contracts, immutable versioning and branch overrides. Every step traced to its RMS BRS requirement ID.

How to use: Work the scenarios top to bottom against a tenant loaded with `--with-demo=all`. Each step gives you a precondition, an action, the expected result, and a **✓ checkpoint** to tick. Green = happy path; amber/red steps deliberately trip a guard to confirm it holds.

01 What Pricing does & how to read this guide

The **pricing** module is the RMS — the single authoritative source for every commercial price. A `pricing.rate` is a priced offering (buy or sell) for a service on a lane; charge lines (`pricing.rate.line`) carry the components, optionally per **leg segment**. Rates group under `pricing.contract` , are sourced ad-hoc via `pricing.spot.request` , and every amendment forks an immutable `pricing.rate.version` . Status is `workflow=True` — transitions are owned by `foundation.workflow` rows, not free writes.

30

DOMAIN COMMANDS

3

TEST SCENARIOS

3

WORKFLOW STATE MACHINES

4

LEG-SEGMENT TAGS

The three state machines you will observe

Rate, contract and spot-request each carry a **governed lifecycle** on a `workflow=True` status field. Watch the statusbar on each form header.

```
rate      : draft → pending_approval → approved → suspended | expired → archived (↘ rejected)
contract: draft → active → suspended ≠ reactivate | expired | terminated
spot      : pending → responded → selected | expired | cancelled
```

Two orthogonal classifications set at rate creation

RATE_TYPE & RATE_SOURCE

- `rate_type` — `buy` (vendor cost) / `sell` (customer price)
- `rate_source` — `contract` · `tariff` · `spot` · `promotional` · `agent` · `manual` · `api_feed`

MARGIN_RISK_LEVEL (PREDICTIVE)

- `safe` ≥ 8% · `watch` 5–8% · `risk` < 5%
- `negative_risk` — margin amount < 0 (billing below cost)
- Thresholds config-driven (PM-008/PM-010); defaults 5% / 8%

The scenarios in this guide

Scenario	What it proves	Key commands	Section
A — Buy/Sell rate lifecycle	Create → line/leg-segment charges → margin & predictive engine → submit → maker-checker approval → approved; margin gate + guards	<code>submit</code> , <code>approve</code> , <code>amend</code>	\$02 · \$03
B — Spot RFQ	Multi-vendor RFQ → responses → compare → award → rate creation; expiry & cancel guards	<code>respond</code> , <code>compare</code> , <code>award</code>	\$04
C — Contract & versioning	Contract lifecycle + cascade amend → version compare/rollback → branch override & publish; immutability guards	<code>contract.amend</code> , <code>version.rollback</code>	\$05 · \$06

Demo fixtures you start from (loaded by `--with-demo=all`)

Forwarder **Acme Forwarding — Mumbai**; customer **Globex Inc.**; carrier **Maersk Demo Line**; lane **INMUM** → **SGSIN**. The seeded sell rate `demo_rate_sell_sea_globex_mbai_sg` (ocean freight + documentation) and the matching buy rate `demo_rate_buy_sea_maersk_mbai_sg` (freight + BAF + a `leg_segment=origin` THC line) both sit on the Globex lane; contract `demo_contract_maersk_sea` groups the buy side. Approvers: `demo_user_ops_manager` (pricing approver), `demo_user_finance` (high-value).

02 Buy/Sell rate lifecycle — creation to approved

Goal: price a new Globex **sell** rate on INMUM → SGSIN, watch the margin engine light up, and drive it through maker-checker approval to **approved**. **Role:** pricing executive (maker) + `demo_user_ops_manager` (checker). **Start:** a fresh draft on the demo lane.

#	Action	Expected result	✓ Checkpoint	Trace
1	Create a <code>pricing.rate</code> with <code>rate_type=sell</code> , <code>rate_source=tariff</code> , mode Sea, currency USD, <code>valid_from/valid_to</code> set.	Rate created in draft ; the <code>pre.pricing.rate.create</code> hook auto-stamps a number in format <code>RATE-YYYY-NNNNNN</code> immediately.	<code>rate_number</code> populated at birth, not blank.	RM-001 RM-005
2	Add charge lines — base ocean freight (<code>leg_segment=main_leg</code>), documentation (<code>all</code>), a THC line (<code>origin</code>) — each with a <code>unit_rate</code> and a <code>minimum_selling_amount</code> .	Lines persist with their leg segment; <code>calculated_sell_amount</code> auto-recomputes (FX-converted to header currency) via the compute runtime.	Sell amount = Σ line rates; leg tags stored.	CHG-001 CAL-001
3	Read <code>calculated_margin_percent</code> and <code>margin_risk_level</code> on the header.	Margin % and the four-bucket risk (<code>safe</code> / <code>watch</code> / <code>risk</code> / <code>negative_risk</code>) derive automatically from the thresholds (§03).	Risk badge matches the margin band.	MAR-001 PM-004
4	Refresh predictive margin (<code>pricing.rate.recalculate_predictive_margin</code>) — also fires automatically via post-create / line-CRUD hooks.	<code>PredictiveMarginService</code> writes <code>predictive_margin_amount / _percent</code> + <code>risk_reason</code> (FX exposure, volatile surcharge, expiry proximity).	risk_reason is human-readable, not blank.	PM-004 PM-005
5	Run Submit (<code>pricing.rate.submit</code>) on a risky -margin rate.	Rate flips draft → pending_approval ; the margin rule (<code>margin_risk_level</code> in (<code>'risk'</code> , <code>'negative_risk'</code>)) routes an <code>ir.approval.case</code> to role <code>pricing.approver</code> ; submitter + approver notified.	Status pending_approval; approval task visible to ops_manager.	WF-001 BR-009
6	As <code>demo_user_ops_manager</code> run Approve (<code>pricing.rate.approve</code>).	Maker-checker passes (checker ≠ owner) → approved . A safe/watch rate instead auto-approves on submit via the catch-all rule.	Status = approved; rate now quotable.	WF-001 WF-002

Multi-leg emphasis — `leg_segment` on every charge line

Each `pricing.rate.line` carries `leg_segment ∈ origin · main_leg · destination · all` (default `all`). The demo buy rate's **THC** line is tagged `origin` and the Stark sell rate's THC is tagged `destination` — so a single rate prices distinct charges per journey leg without duplicating the header. Confirm the tag survives a save and appears in the version snapshot (§05).

✓ Scenario A pass criteria

The rate reached **approved** with a system-composed `RATE-YYYY-NNNNNN` number, an auto-computed sell amount / margin / risk badge, a predictive-margin reason, and a maker-checker approval that the owner could not self-approve.

03 The margin engine & the guards that must hold

Half of testing pricing is confirming it **refuses** bad input. First the margin classification thresholds; then eight negative checks that each trip a specific `ValueError` (or `PermissionError`).

Margin classification thresholds (defaults; config-driven per PM-008/PM-010)

Band	Condition	Routing effect on submit
safe	margin % ≥ 8.0	Auto-approve rule (priority 10) → approved on submit.
watch	5.0 ≤ margin % < 8.0	Auto-approve — but visibly flagged for review.
risk	margin % < 5.0	Risk rule (priority 30) → pending_approval to <code>pricing.approver</code> .
negative_risk	margin amount < 0	Routes to approval regardless of percent — needs senior sign-off (BR-010).

Negative checks — each must be refused with the real message

#	Action	Expected refusal (verbatim)	✓	Trace
1	<code>submit</code> a rate with no charge lines .	"Rate must have at least one charge line before submission."	Blocked	RM-001
2	<code>submit</code> a rate whose <code>valid_to</code> is already past.	"Rate validity period has already expired — extend the validity window (<code>valid_to</code>) before submitting this rate for approval."	Blocked	RM-009
3	<code>submit</code> a rate whose key fields + lines duplicate an active rate.	"A rate with identical key fields already exists: <RATE-Nº>. Duplicate rates are not allowed ..."	Blocked	BR-015
4	Rate owner runs <code>approve</code> on their own rate.	"Maker-checker violation: you cannot approve your own rate."	Blocked	WF-001
5	Edit a locked header field (e.g. <code>currency_id</code> , <code>mode_id</code> , <code>valid_to</code>) on a pending/approved rate.	"Rate is locked while its approval case is PENDING or APPROVED — use Amend (Phase 2) to fork a new version or Suspend ..."	Blocked	BR-011
6	Create / edit / delete a charge line under a locked rate.	"Cannot modify pricing.rate.line — parent rate's approval case is PENDING/APPROVED. Use Amend (Phase 2) on the parent rate or Suspend it."	Blocked	BR-011
7	Write a line where <code>unit_rate</code> < <code>minimum_selling_amount</code> .	"unit_rate <v> is below the minimum selling amount <msp>."	Blocked	MAR-004
8	<code>suspend</code> a rate that is still draft (not approved).	"Only approved rates can be suspended."	Blocked	WF-013

⚠ Amend is the escape hatch, not a header edit

To change a locked rate, run `pricing.rate.amend` (reason mandatory — omitting it raises "amendment_reason is required when amending a rate (BR-017, VER-003)"). Amend Forks a `pricing.rate.version` snapshot and resets the rate to **draft** for re-editing. Only **approved** or **suspended** rates may be amended; `archive` is the terminal end-of-life move (status `archived`, `active=False`).

04 Spot RFQ — multi-vendor enquiry to awarded rate

Goal: raise a one-off buy RFQ on the Globex lane, collect competing vendor responses, compare against standing tariffs, and award the winner — which auto-creates a `rate_source=spot` rate. **Role:** pricing desk. **Start:** a fresh `pricing.spot.request`.

pending → responded → selected | expired | cancelled (select/award only from responded)
each response = one `pricing.spot.response` (vendor · rate_amount · currency · rate_expiry_at · is_awarded)

#	Action	Expected result	✓ Checkpoint	Trace
1	Create a <code>pricing.spot.request</code> with <code>request_type=buy</code> , mode Sea, origin/dest set, vendor = Maersk.	Created in pending ; pre-create stamps <code>SPT-YYYY-NNNNNN</code> ; the post-create hook opens an <code>activity.log</code> TASK and a <code>mail.outbox</code> email to the vendor (skipped if no email on file).	request_number set; RFQ activity + outbox row present.	SPT-001 SPT-003
2	Record a vendor reply via <code>pricing.spot.request.respond</code> (vendor_id, provider_rate_amount, currency, rate_expiry_at).	Status pending → responded ; a <code>pricing.spot.response</code> child is created; <code>response_count</code> / <code>best_quote_amount</code> auto-recompute.	1 response child; count = 1.	SPT-011
3	<code>respond</code> again with a second vendor's (cheaper) quote.	Stays responded ; second child added; <code>best_quote_amount</code> drops to the lower figure.	count = 2; best = min(quotes).	SPT-011
4	Run <code>pricing.spot.request.compare</code> .	Returns matching active contracts + approved tariff rates on the same lane/mode, sorted by amount — spot vs standing price side by side.	Comparison list returned, cheapest first.	SPT-008
5	Run <code>pricing.spot.request.award</code> with the winning <code>awarded_response_id</code> .	Response marked <code>is_awarded</code> ; siblings cleared; status responded → selected ; a <code>pricing.rate</code> (<code>rate_source=spot</code>) + one line is created, linked via <code>linked_rate_id</code> , and auto-submitted.	selected; linked spot rate exists.	SPT-006 BR-008

Negative checks — award & expiry guards

#	Action	Expected refusal (verbatim)	✓ / Trace
6	<code>award</code> without <code>awarded_response_id</code> .	"'awarded_response_id' is required — please select one vendor response to award."	Blocked · SPT-011
7	<code>select</code> when responses exist but none is awarded.	"Multiple vendor responses exist. Please select one response as 'Awarded' using the 'Award & Select' button ..."	Blocked · SPT-011
8	<code>select</code> / award an expired spot request (or an expired quoted rate).	"Expired spot request cannot be selected or reused (BR-022)." / "The selected spot rate has expired and cannot be selected (BR-022)."	Blocked · BR-022
9	<code>expire</code> then (on another) <code>cancel</code> a request.	Status lands in terminal expired / cancelled respectively (from pending or responded).	Terminal · SPT-003

✓ Scenario B pass criteria

An RFQ went pending → responded with ≥2 collated vendor responses, compared against standing rates, and awarded a winner that produced an auto-submitted `spot` rate — with the two award guards and the expiry guard all confirmed to refuse.

05 Contract lifecycle & rate versioning

Goal: drive a contract through its lifecycle, cascade an amendment onto its linked rates, then compare and roll back rate versions. **Start:** contract `demo_contract_maersk_sea` with its linked buy rate. **Role:** contract owner + pricing manager.

#	Action	Expected result	✓ Checkpoint	Trace
1	Run <code>pricing.contract.activate</code> on a draft contract.	Status draft → active ; its customer/lane/mode rates now take precedence over general tariffs in rate search.	Contract active; overrides tariff.	CON-001 BR-006
2	Run <code>pricing.contract.amend</code> with an <code>amendment_reason</code> .	Every linked approved/suspended rate is auto-amended (forking a version each) and the contract is reset to draft for re-approval.	amended_rates_count > 0; contract draft.	CON-008
3	Confirm the cascade produced a <code>pricing.rate.version</code> v1 snapshot on the amended rate (header + lines + slabs as JSON).	Version row created, <code>version_number=1</code> , mandatory <code>amendment_reason</code> stored, rate reset to draft .	v1 snapshot present & complete.	VER-001 VER-003
4	Edit the draft rate's freight line (a GRI), re-submit & approve, then amend again → v2.	v2 snapshot captured; <code>change_summary</code> auto-describes the delta (header fields + +/- lines changed).	v2 exists; change_summary populated.	VER-005
5	Run <code>pricing.rate.version.compare</code> on v2 (defaults to preceding version).	Field-level diff old → new: <code>header_changes</code> + <code>line_changes</code> (added / removed / changed), lines aligned by product + sequence.	Diff shows the freight-rate delta.	VER-008
6	Run <code>pricing.rate.version.rollback</code> targeting v1.	Current state is first snapshot (a fresh version), then v1's header + lines are restored onto the now- draft rate; it re-enters approval on next submit. History never mutated.	Rate matches v1; status draft.	VER-009
7	Exercise the rest of the contract machine: <code>suspend</code> → <code>reactivate</code> → <code>terminate</code> ; separately <code>renew</code> .	active → suspended → active → terminated as commanded; <code>renew</code> creates a fresh draft contract (+1yr) and copies rates, marking the old one <code>renewal_status=renewed</code> .	Each transition lands correctly.	CON-007

⚠ Rollback needs a live rate to land on

`version.rollback` only runs when the parent rate is **approved** or **suspended** — on any other status it raises “Only approved or suspended rates can be rolled back (current status: ...)”. It reuses the `amend` path, so the mandatory-reason and snapshot rules from §03 all apply. Branch visibility (`branch_ids`) is deliberately *not* rolled back — that is governed only by publish-to-branches (§06).

06 Branch overrides, publish & immutability guards

Goal: prove head-office publish-to-branch and branch-local overrides are privileged, and that version history is permanently immutable.

Roles: pricing manager (holds `pricing.rate:publish_to_branches`) vs an ordinary user.

#	Action	Expected result / refusal (verbatim)	✓	Trace
1	Run <code>pricing.rate.create_branch_override</code> with a <code>branch_id</code> on a global/parent rate.	A branch-scoped draft clone is created, linked via <code>override_of_rate_id</code> , <code>branch_ids=[branch]</code> — deliberately not auto-submitted (a verbatim clone shares the parent's dedupe hash).	Override draft linked to parent.	ME-007
2	<code>create_branch_override</code> with no <code>branch_id</code> .	"branch_id is required to create a branch tariff override."	Blocked	ME-007
3	Override a rate scoped to <i>other</i> branches while <code>pricing.branch_sharing_mode=strict</code> .	"Inter-branch sharing is strict: this branch cannot override a rate scoped to other branches. Set <code>pricing.branch_sharing_mode=permissive</code> to allow it."	Blocked	ME-008
4	As pricing manager run <code>pricing.rate.publish_to_branches</code> with a branch list.	The rate's <code>branch_ids</code> M2M is set to the supplied orgs (empty list = global to all branches).	Visibility updated.	ME-006
5	As an ordinary user, try a plain <code>ede.update</code> writing <code>branch_ids</code> directly.	Vetoed: "Updating <code>branch_ids</code> requires <code>pricing.rate:publish_to_branches</code> — use the <code>pricing.rate.publish_to_branches</code> command instead." (unauthorised publish also raises "User lacks <code>pricing.rate:publish_to_branches</code> ...")	Blocked	ME-006
6	Attempt to update an approved <code>pricing.rate.version</code> snapshot.	" <code>pricing.rate.version</code> is immutable once approved — use <code>pricing.rate.amend</code> to create a new version instead of editing the historical snapshot."	Blocked	VER-001
7	Attempt to <code>delete</code> any <code>pricing.rate.version</code> record.	" <code>pricing.rate.version</code> records cannot be deleted — version history is permanent for audit purposes (VER-001)."	Blocked	VER-001

Config toggles & the renewal sweep worth verifying

Config key / job	Default	Effect to verify
<code>pricing.branch_sharing_mode</code>	strict	strict → override only global / already-visible rates; permissive → any branch may override any rate (step 3).
<code>pricing.margin.threshold_*</code>	5% / 8%	Shift the risk / watch bands (PM-008/PM-010) → changes which rates route to approval (§03).
<code>pricing.rate.renewal_sweep</code>	cron 0 2 * * *	Daily job: rates inside <code>days_before_expiry</code> get <code>renewal_status=due</code> + reminder; auto-suspends lapsed rates; contract volume alerts at 80/90/100% bands.

✓ Full-module pass criteria

All three scenarios green: rate lifecycle with margin engine + maker-checker (A), multi-vendor spot RFQ to awarded rate (B), and contract cascade + version compare/rollback + branch override/publish (C) — with every negative guard across §03, §04 and §06 confirmed to refuse with its real message.

Traceability: RM-/CHG-/MAR-/CAL-/CUR-/WF-/VER-/SPT-/CON-/REN-/PM-/ME-/BR- IDs map to the RMS BRS v1.0 via `roadmap/logistics/pricing/sow.md` (§2, §5) and `tds.md`. Cross-module effects (quote consumption via `pricing.rate.usage_check_requested` event, job costing) are verified in the sales-crm and shipment guides.